MAT 3007 – Optimization

Solve Integer Programs

Lecture 27

Instructor: Junchi Yang

CUHK-Shenzhen

Announcements

Final Exam:

- Friday, May 16th, 13:30 - 15:30 pm.
- Total Time: 120 minutes.
- You can bring two self-made sheets of A4 paper (with arbitrary notes on both sides of it).
- Coverage: mainly contents after midterm (excluding interior point methods) (L-16 – L-29).
- Will post a sample final.

Recap

Integer Problems:

$$\begin{aligned} \text{minimize}_{\boldsymbol{x}} \quad & \boldsymbol{c}^\top \boldsymbol{x} \\ \text{subject to} \quad & \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b} \\ & \boldsymbol{x} \geq \boldsymbol{0} \\ & \boldsymbol{x} \in \mathbb{Z}^n \end{aligned}$$

Applications:

- production planning, Knapsack problem, matching problems, ...

Example: Knapsack Problem

John is planning a trip. There are n items he would like to bring.

- The ith item has value v_i.
- The weight of ith item is a_i.
- His bag has a maximum allowable weight C.
- He wants to bring as much value as possible.

Example: Knapsack Problem

John is planning a trip. There are n items he would like to bring.

- The ith item has value v_i.
- The weight of ith item is a_i.
- His bag has a maximum allowable weight C.
- He wants to bring as much value as possible.

Decision Variables:

- x_i: whether to bring ith item or not: $x_i \in \{0, 1\}$.

Example: Knapsack Problem

John is planning a trip. There are n items he would like to bring.

- The ith item has value v_i.
- The weight of ith item is a_i.
- His bag has a maximum allowable weight C.
- He wants to bring as much value as possible.

Decision Variables:

- x_i: whether to bring ith item or not: $x_i \in \{0, 1\}$.

Optimization Problem:

$$\begin{aligned} \text{maximize}_{\boldsymbol{x}} \quad & \sum_{i=1}^{n} v_i x_i \\ \text{subject to} \quad & \sum_{i=1}^{n} a_i x_i \leq C \\ & x_i \in \{0, 1\} \quad \forall\ i. \end{aligned}$$

What if he can bring at most 3 items ?

$\sum_i x_i \leq 3$

$\rightsquigarrow$ This is a binary optimization problem.

Logical Constraints

Integer variables can be used to model logical constraints.

- Binary variables $\{0, 1\}$ correspond to true or false decisions.
- For $x, y \in \{0, 1\}$, the constraint "$x + y = 1$" means: "only one of the variables x or y is 1/true".

Logical Constraints

Integer variables can be used to model logical constraints.

- Binary variables $\{0, 1\}$ correspond to true or false decisions.
- For $x, y \in \{0, 1\}$, the constraint "$x + y = 1$" means: "only one of the variables x or y is 1/true".
- Suppose we have m many constraints: $g_i(x) \leq 0$, $i = 1, \ldots, m$ and at least k many of those constraints should be satisfied.

Logical Constraints

Integer variables can be used to model logical constraints.

- Binary variables $\{0, 1\}$ correspond to true or false decisions.
- For $x, y \in \{0, 1\}$, the constraint "$x + y = 1$" means: "only one of the variables x or y is 1/true".
- Suppose we have m many constraints: $g_i(x) \leq 0$, $i = 1, \ldots, m$ and at least k many of those constraints should be satisfied.

$\rightsquigarrow$ We introduce $y_i \in \{0, 1\}$ and modify the constraints to

$$g_1(x) \leq My_1, \quad \ldots \quad g_m(x) \leq My_m, \quad \sum_{i=1}^{m} (1 - y_i) \geq k,$$

where M is a sufficiently large parameter.

at least k of y_i should be 0.

- If $g_i(x) > 0$, then $y_i = 1$. Otherwise, if $g_i(x) \leq 0$, we can choose $y_i \in \{0, 1\}$.

if $y_i = 0$, $g_i(x) \leq 0$

Solving Integer Programs

Our Objective

We want to answer two questions in our study:

- How different is it from linear optimization problems? Is it more difficult?
- How to solve IPs?

Our Objective

We want to answer two questions in our study:

- How different is it from linear optimization problems? Is it more difficult?
- How to solve IPs?

We can give a quick answer to the first question:

- Integer problems are NP-hard. It is unlikely to have polynomial-time algorithms.
- Even to verify if a set (polyhedron) contains an integer point is very hard.
- In contrast, we have polynomial-time algorithm for linear optimization, such as ellipsoid algorithm and interior point method.

An Intuitive Thought: Relaxation

General Idea:

We can first relax the integer constraints, solve the linear program and find some integer solution near the optimal solution of the linear program.

An Intuitive Thought: Relaxation

General Idea:

We can first relax the integer constraints, solve the linear program and find some integer solution near the optimal solution of the linear program.

- We call the corresponding linear program the LP relaxation of the IP.
- May work well in some case.
- But may fail in other cases.

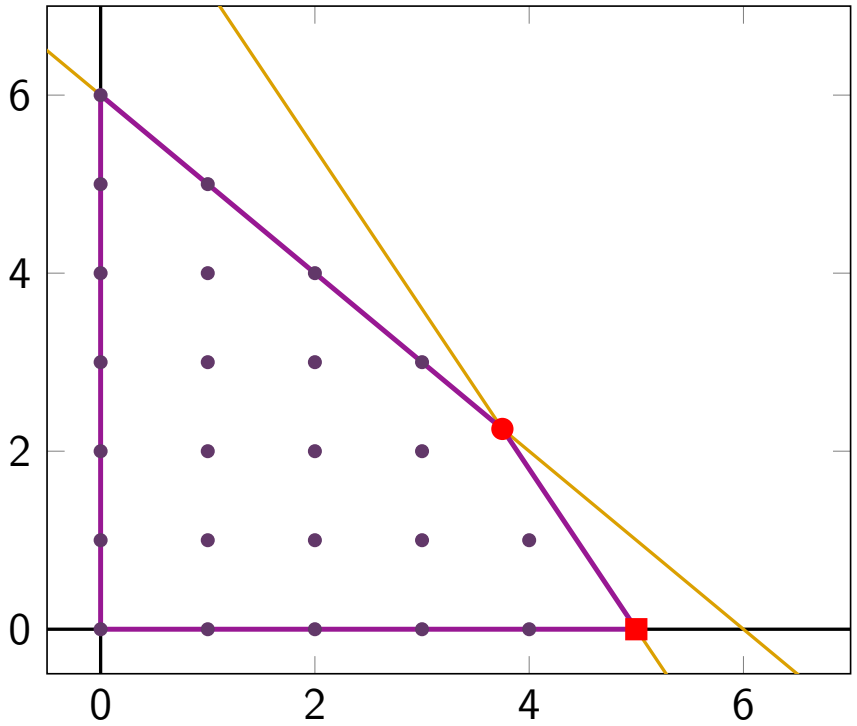
Example: Relaxation

Consider the following integer program (a production planning problem):

$$\begin{aligned} \text{maximize} \quad & 8x_1 + 5x_2 \\ \text{subject to} \quad & 9x_1 + 5x_2 \leq 45 \\ & x_1 + x_2 \leq 6 \\ & x_1, x_2 \geq 0 \\ & x_1, x_2 \in \mathbb{Z} \end{aligned}$$

- If we consider the LP relaxation, the optimal solution is $(x_1, x_2) = (15/4, 9/4)$.

Example: Relaxation



- If we round it to the nearest integer solution, we get $(4, 2)$ which is not feasible for the integer program.
- If we consider the nearest feasible integer point, it will be $(3, 2)$, the objective value is 34.
- The optimal solution to the integer program is actually $(5, 0)$, the objective value is 40.

Another Example: Knapsack Problem

There are three items:

- Item A: 3kg with value 300. x_1
- Item B: 2kg with value 180. x_2
- Item C: 2kg with value 170. x_3

The capacity of the bag is 4kg.

$$\max \ 300x_1 + 180x_2 + 170x_3$$

$$\text{s.t} \ \ 3x_1 + 2x_2 + 2x_3 \leq 4$$

$$x_1, x_2, x_3 \in \{0, 1\}.$$

Another Example: Knapsack Problem

There are three items:

- Item A: 3kg with value 300.
- Item B: 2kg with value 180.
- Item C: 2kg with value 170.

The capacity of the bag is 4kg.

Two messages: (max problem)

① relaxation provides an upper bound

② feasible integer solution provides a lower bound.

- The relaxation of a binary constraint $x \in \{0, 1\}$ is $x \in [0, 1]$.
- Optimal solution to LP relaxation: (1, 1/2, 0) with optimal value 390.
- Nearest feasible integer solution: (1, 0, 0) with value 300.
- Optimal integer solution: (0, 1, 1) with value 350.

Summary

The optimal solution to the integer program might not be close to the optimal solution to the LP relaxation.

- Directly rounding the LP solution to integers may not yield a good solution ($\rightsquigarrow$ might even be infeasible).
- The reason is the very combinatorial nature of the IP problem (no continuity in the solution space).

But does the LP relaxation provide any useful information?

LP Relaxation as a Bound for IP

Theorem: LP Relaxation as a Bound for IP

1. For a maximization problem, the optimal value of the LP relaxation provides an upper bound for the optimal value of the IP.

Why? Because we consider a larger feasible region.

LP Relaxation as a Bound for IP

Theorem: LP Relaxation as a Bound for IP

1. For a maximization problem, the optimal value of the LP relaxation provides an upper bound for the optimal value of the IP.
2. For a minimization problem, the optimal value of the LP relaxation provides a lower bound for the optimal value of the IP.

LP Relaxation as a Bound for IP

Theorem: LP Relaxation as a Bound for IP

1. For a maximization problem, the optimal value of the LP relaxation provides an upper bound for the optimal value of the IP.
2. For a minimization problem, the optimal value of the LP relaxation provides a lower bound for the optimal value of the IP.

The difference between the optimal value of the LP and the IP is called the integrality gap.

- For maximization problems, the integrality gap is $v^{LP} - v^{IP}$. ≥ 0
- For minimization problems, the integrality gap is $v^{IP} - v^{LP}$. ≥ 0

v^{LP}: optimal value for LP relaxation. v^{IP}: optimal value for IP.

Use LP Relaxation as a Bound

Maximization Problem: Suppose we solved the LP relaxation and obtain the optimal value is v^{LP} (with some fractional solution).

$\rightsquigarrow$ Then we round the solution to a feasible integer point and find the objective value is $v^{Rounding}$.

Use LP Relaxation as a Bound

Maximization Problem: Suppose we solved the LP relaxation and obtain the optimal value is v^{LP} (with some fractional solution).

$\rightsquigarrow$ Then we round the solution to a feasible integer point and find the objective value is $v^{Rounding}$.

The difference between the rounded solution and the optimal integer solution (whose objective value is v^{IP}) satisfies:

$$0 \leq v^{IP} - v^{Rounding} \leq v^{LP} - v^{Rounding}$$

Because this point is feasible (integer), then for maximization,

$$v^{Rounding} \leq v^{IP}$$

From previous slide, $v^{LP} \geq v^{IP}$

Use LP Relaxation as a Bound

Maximization Problem: Suppose we solved the LP relaxation and obtain the optimal value is v^{LP} (with some fractional solution).

- $\rightsquigarrow$ Then we round the solution to a feasible integer point and find the objective value is $v^{Rounding}$.

The difference between the rounded solution and the optimal integer solution (whose objective value is v^{IP}) satisfies:

$$0 \leq v^{IP} - v^{Rounding} \leq v^{LP} - v^{Rounding}$$

That is, one can use the LP relaxation solution to construct a bound on how good a certain feasible integer point is.

For minimization, $v^{LP} \leq v^{IP}$ and $v^{Rounding} \geq v^{IP}$

$\Rightarrow$ $v^{Rounding} - v^{LP} \geq v^{Rounding} - v^{IP} \geq 0$

The Ideal Case

There are some cases where solving the LP relaxation already solves the integer problem.

Theorem

If the optimal solution to the LP relaxation is integer, then the solution must be optimal to the IP problem.

- Why? Since $v^{LP} - v^{Rounding} = 0$, we must have $v^{IP} - v^{Rounding} = 0$.

Total Unimodularity

LPs and Integer Solutions

Recap: For LPs, if an optimal solution exists, there must exist an optimal solution that is a Basic Feasible Solution (BFS).

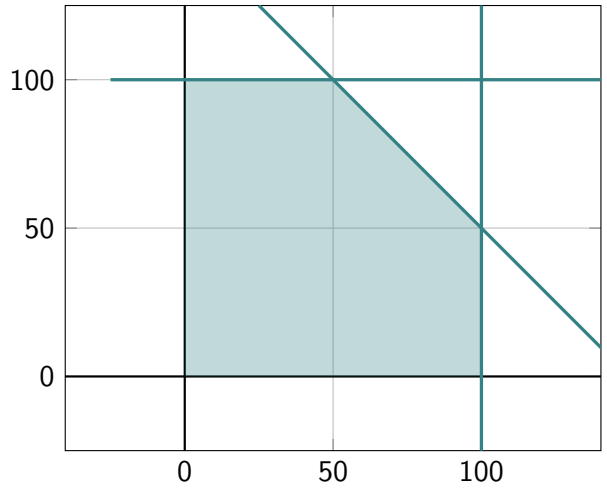
- If every BFS is an integer point, then the LP must have an integer optimal solution.

LPs and Integer Solutions

Recap: For LPs, if an optimal solution exists, there must exist an optimal solution that is a Basic Feasible Solution (BFS).

- If every BFS is an integer point, then the LP must have an integer optimal solution.

For example, in the following production planning problem:

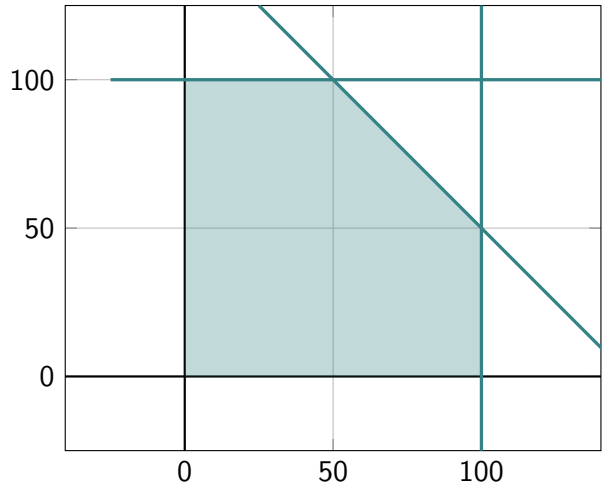


LPs and Integer Solutions

Recap: For LPs, if an optimal solution exists, there must exist an optimal solution that is a Basic Feasible Solution (BFS).

- If every BFS is an integer point, then the LP must have an integer optimal solution.

For example, in the following production planning problem:



All extreme points are integers. Therefore, there must exist an optimal solution that is an integer point. (This criterion has nothing to do with the objective function.)

Total Unimodularity

We introduce a condition under which all BFS must be integers — total unimodularity.

Definition: Total Unimodularity

A matrix $\boldsymbol{A}$ is said to be totally unimodular if the determinant of each submatrix of $\boldsymbol{A}$ is either 0, 1, or -1.

Total Unimodularity

We introduce a condition under which all BFS must be integers — total unimodularity.

Definition: Total Unimodularity

A matrix $\boldsymbol{A}$ is said to be totally unimodular if the determinant of each submatrix of $\boldsymbol{A}$ is either 0, 1, or -1.

Theorem: Total Unimodularity and Integer Solutions

If the constraint matrix $\boldsymbol{A}$ is totally unimodular and $\boldsymbol{b}$ is an integer vector, then all the BFS for the LP relaxation are integers. If it has an optimal solution, then it must have an optimal solution that is an integer solution.

Total Unimodularity

We introduce a condition under which all BFS must be integers — total unimodularity.

Definition: Total Unimodularity

A matrix $\boldsymbol{A}$ is said to be totally unimodular if the determinant of each submatrix of $\boldsymbol{A}$ is either 0, 1, or -1.

Theorem: Total Unimodularity and Integer Solutions

If the constraint matrix $\boldsymbol{A}$ is totally unimodular and $\boldsymbol{b}$ is an integer vector, then all the BFS for the LP relaxation are integers. If it has an optimal solution, then it must have an optimal solution that is an integer solution.

Total Unimodularity (TU):

- In order for $\boldsymbol{A}$ to be TU, it can only have 0, 1 or -1 entries.
- But it requires much more...

One Sufficient Condition for TU

Theorem (Sufficient condition)

Let $\boldsymbol{A}$ be an $m \times n$ matrix. Then the following conditions together are sufficient for $\boldsymbol{A}$ to be totally unimodular:

1. Every column of $\boldsymbol{A}$ contains at most two non-zero entries;
2. Every entry in $\boldsymbol{A}$ is 0, $+1$, or -1;
3. The rows of $\boldsymbol{A}$ can be partitioned into two disjoint sets B and C such that:
   - If two non-zero entries in a column of $\boldsymbol{A}$ have the same sign, then the row of one entry is in B, and the row of the other in C;
   - If two non-zero entries in a column of $\boldsymbol{A}$ have opposite signs, then the rows are both in B, or both in C.

Example: Matching Problem

Recall the matching problem:

$$\begin{aligned} \text{maximize} \quad & \sum_{i=1}^{n} \sum_{j=1}^{n} v_{ij} x_{ij} & \\ \text{s.t.} \quad & \sum_{i=1}^{n} x_{ij} = 1 & \forall\ j \\ & \sum_{j=1}^{n} x_{ij} = 1 & \forall\ i \\ & x_{ij} \in \{0, 1\} & \end{aligned}$$

The constraint matrix is TU.

Example: Matching Problem

Recall the matching problem:

$$\begin{aligned} \text{maximize} \quad & \sum_{i=1}^{n} \sum_{j=1}^{n} v_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{i=1}^{n} x_{ij} = 1 \qquad \forall\, j \\ & \sum_{j=1}^{n} x_{ij} = 1 \qquad \forall\, i \\ & x_{ij} \in \{0, 1\} \end{aligned}$$

The constraint matrix is TU.

An example when $n = 2$ (in the order of $x_{11}, x_{12}, x_{21}, x_{22}$):

$$\boldsymbol{A} = \begin{array}{c} \\ B \\ B \\ C \\ C \end{array} \begin{array}{c} \begin{array}{cccc} \downarrow & \downarrow & \downarrow & \downarrow \end{array} \\ \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \end{array} \qquad \begin{pmatrix} x_{11} \\ x_{12} \\ x_{21} \\ x_{22} \end{pmatrix} \begin{array}{l} \rightarrow x_{11} + x_{21} = 1 \\ \rightarrow x_{12} + x_{21} = 1 \\ \rightarrow x_{11} + x_{12} = 1 \\ \rightarrow x_{21} + x_{22} = 1 \end{array}$$

Choosing $B = \{1, 2\}$, $C = \{3, 4\}$, the condition holds and $\boldsymbol{A}$ is TU.

Matching Problem: Observation

Therefore, to solve the matching problem, we can safely relax the binary constraints to $0 \leq x_{ij} \leq 1$.

It is guaranteed that there exists an integer optimal solution of the LP relaxation (and the simplex method can find it).

- This result also applies to transportation problems and some network flow problems.

Another Example

Consider the following matrix $\boldsymbol{A}$:

$$\boldsymbol{A} = \begin{bmatrix} -1 & -1 & 0 & 0 & 0 & 1 \\ 1 & 0 & -1 & -1 & 0 & 0 \\ 0 & 1 & 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 & 1 & -1 \end{bmatrix}$$

Then we can make the partition to be $B = \{1, 2, 3, 4\}$, $C = \emptyset$. The condition holds and $\boldsymbol{A}$ is TU.

Discussion

The TU property is uncommon in practice.

$\rightsquigarrow$ If one solves the LP relaxation and if the optimal BFS is not an integer point, it means that the TU property does not hold.

- We need to have methods to solve IP systematically (and efficiently).
- The most commonly applied technique is the branch-and-bound method.

Branch-and-Bound Method

The General Idea

Consider the following example:

$$\begin{aligned} \text{maximize} \quad & 8x_1 + 5x_2 \\ \text{subject to} \quad & 9x_1 + 5x_2 \leq 45 \\ & x_1 + x_2 \leq 6 \\ & x_1, x_2 \geq 0, \quad x_1, x_2 \in \mathbb{Z} \end{aligned}$$

- We solve the LP relaxation and get $\boldsymbol{x}^* = (15/4, 9/4)$.
- x_1 is not an integer (we can not allow $15/4$ in the solution).

The General Idea

Consider the following example:

$$\begin{aligned} \text{maximize} \quad & 8x_1 + 5x_2 \\ \text{subject to} \quad & 9x_1 + 5x_2 \leq 45 \\ & x_1 + x_2 \leq 6 \\ & x_1, x_2 \geq 0, \quad x_1, x_2 \in \mathbb{Z} \end{aligned}$$

- We solve the LP relaxation and get $\boldsymbol{x}^* = (15/4, 9/4)$.
- x_1 is not an integer (we can not allow $15/4$ in the solution).

Solution: Consider two new subproblems:

- One with an additional constraint $x_1 \leq 3$.
- One with an additional constraint $x_1 \geq 4$.

The General Idea

Consider the following example:

$$\begin{aligned} \text{maximize} \quad & 8x_1 + 5x_2 \\ \text{subject to} \quad & 9x_1 + 5x_2 \leq 45 \\ & x_1 + x_2 \leq 6 \\ & x_1, x_2 \geq 0, \quad x_1, x_2 \in \mathbb{Z} \end{aligned}$$

- We solve the LP relaxation and get $\boldsymbol{x}^* = (15/4, 9/4)$.
- x_1 is not an integer (we can not allow $15/4$ in the solution).

Solution: Consider two new subproblems:

- One with an additional constraint $x_1 \leq 3$.
- One with an additional constraint $x_1 \geq 4$.

The optimal solution to the IP must still be in one of the subproblems, but solutions with $3 < x_1 < 4$ are eliminated.

The General Process

First, we solve the LP relaxation of the IP:

- If the solution is an integer, then it is optimal to the IP.

The General Process

First, we solve the LP relaxation of the IP:

- If the solution is an integer, then it is optimal to the IP.

If the optimal solution to the LP relaxation is $\boldsymbol{x}^*$ and $x_i^* \notin \mathbb{Z}$, then branch the problem into the following two:

1. One with an added constraint $x_i \leq \lfloor x_i^* \rfloor$, we call this (S1);
2. One with an added constraint $x_i \geq \lceil x_i^* \rceil$, we call this (S2),

where $\lfloor \cdot \rfloor$ means rounding down, and $\lceil \cdot \rceil$ means rounding up.
Example: $\lfloor 3.75 \rfloor = 3$, $\lceil 3.75 \rceil = 4$.

The General Process

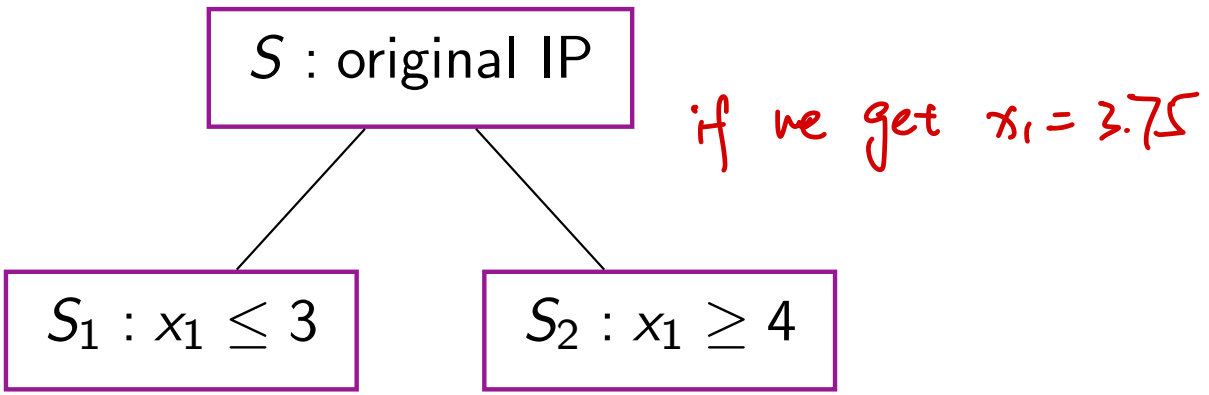
First, we solve the LP relaxation of the IP:

- If the solution is an integer, then it is optimal to the IP.

If the optimal solution to the LP relaxation is $\boldsymbol{x}^*$ and $x_i^* \notin \mathbb{Z}$, then branch the problem into the following two:

1. One with an added constraint $x_i \leq \lfloor x_i^* \rfloor$, we call this (S1);
2. One with an added constraint $x_i \geq \lceil x_i^* \rceil$, we call this (S2),

where $\lfloor \cdot \rfloor$ means rounding down, and $\lceil \cdot \rceil$ means rounding up.
Example: $\lfloor 3.75 \rfloor = 3$, $\lceil 3.75 \rceil = 4$.



What Happens after Branching?

We get two new IP's after branching:

- We then solve (S1) and (S2) and assume we can get the optimal solutions y_1^* and y_2^* with optimal values v_1^* and v_2^* (both (S1) and (S2) are still integer programs).

What Happens after Branching?

We get two new IP's after branching:

- We then solve (S1) and (S2) and assume we can get the optimal solutions y_1^* and y_2^* with optimal values v_1^* and v_2^* (both (S1) and (S2) are still integer programs).

Claim (for maximization)

- If $v_2^* \leq v_1^*$, then y_1^* is the optimal solution to the original IP.
- If $v_1^* \leq v_2^*$, then y_2^* is the optimal solution to the original IP.

The claim is true because the union of the feasible regions in each branch equals the feasible region of the original problem.

What Happens after Branching?

We get two new IP's after branching:

- We then solve (S1) and (S2) and assume we can get the optimal solutions y_1^* and y_2^* with optimal values v_1^* and v_2^* (both (S1) and (S2) are still integer programs).

Claim

- If $v_2^* \leq v_1^*$, then y_1^* is the optimal solution to the original IP.
- If $v_1^* \leq v_2^*$, then y_2^* is the optimal solution to the original IP.

The claim is true because the union of the feasible regions in each branch equals the feasible region of the original problem.

Question: How to solve (S1) and (S2)?

$\rightsquigarrow$ Use the same idea (solve LP relaxation and further branch).

Bounding

For each branch, we can construct an upper bound and a lower bound for the problem (assume we are solving a max. problem):

Bounding

For each branch, we can construct an upper bound and a lower bound for the problem (assume we are solving a max. problem):

- **Upper bound:** The LP relaxation solution will be an upper bound — the objective value of any integer solution from this node must be no greater than than the optimal value of the relaxed LP.

Say we solve LP relaxation for (S_2), then it provides an upper bound for (S_2). not necessarily the original (IP).

Bounding

For each branch, we can construct an upper bound and a lower bound for the problem (assume we are solving a max. problem):

- **Upper bound:** The LP relaxation solution will be an upper bound — the objective value of any integer solution from this node must be no greater than than the optimal value of the relaxed LP.
- **Lower bound:** The objective value of any feasible (integer) point is a lower bound for the optimal value — the optimal solution of the IP must be no less than the objective value achieved by any feasible point.

lower bound for the (IP).

Bounding: Pruning the Branches

Bounding Procedure:

- At a certain node, when the optimal value of the LP relaxation of this branch is even less than the current lower bound, then we can abandon this branch.

Bounding: Pruning the Branches

Bounding Procedure:

- At a certain node, when the optimal value of the LP relaxation of this branch is even less than the current lower bound, then we can abandon this branch.
- These results will be the opposite if we are minimizing.

We will use the bounding steps to prune unnecessary computations (or in other words, remove unnecessary branches).

Example

Consider the earlier example

$$\begin{aligned} \text{maximize} \quad & 8x_1 + 5x_2 \\ \text{subject to} \quad & 9x_1 + 5x_2 \leq 45 \\ & x_1 + x_2 \leq 6 \\ & x_1, x_2 \geq 0, \quad x_1, x_2 \in \mathbb{Z} \end{aligned}$$

- We solve the LP relaxation and get $\boldsymbol{x}^* = (15/4, 9/4)$.
- We first branch for x_1.

We solve two subproblems:

- One with an additional constraint $x_1 \leq 3 \rightsquigarrow$ problem (S1).
- One with an additional constraint $x_1 \geq 4 \rightsquigarrow$ problem (S2).

Example: Continued

Now we solve (S1) and (S2) respectively:

- We solve the LP relaxation of (S1). The optimal solution is $(3, 3)$. This is an integer solution, so we are done with this branch (the optimal value is 39).
- We solve the LP relaxation of (S2), the optimal solution is $(4, 1.8)$ (the optimal value is 41).

Example: Continued

Now we solve (S1) and (S2) respectively:

- We solve the LP relaxation of (S1). The optimal solution is $(3, 3)$. This is an integer solution, so we are done with this branch (the optimal value is 39).
- We solve the LP relaxation of (S2), the optimal solution is $(4, 1.8)$ (the optimal value is 41).

The solution to (S2) is not an integer. We have to do further branching:

- We add a constraint $x_2 \leq 1 \rightsquigarrow$ (S3).
- We add a constraint $x_2 \geq 2 \rightsquigarrow$ (S4).

Example Continued

Subproblem (S3):

$$\begin{aligned} \text{maximize} \quad & 8x_1 + 5x_2 \\ \text{subject to} \quad & 9x_1 + 5x_2 \leq 45 \\ & x_1 + x_2 \leq 6 \\ & x_1 \geq 4, x_2 \leq 1 \quad x_1, x_2 \geq 0, \quad x_1, x_2 \in \mathbb{Z} \end{aligned}$$

Subproblem (S4):

$$\begin{aligned} \text{maximize} \quad & 8x_1 + 5x_2 \\ \text{subject to} \quad & 9x_1 + 5x_2 \leq 45 \\ & x_1 + x_2 \leq 6 \\ & x_1 \geq 4, x_2 \geq 2 \quad x_1, x_2 \geq 0, \quad x_1, x_2 \in \mathbb{Z} \end{aligned}$$

One can easily see that (S4) is not feasible. Thus we do not need to further consider this subproblem.

Solving (S3)

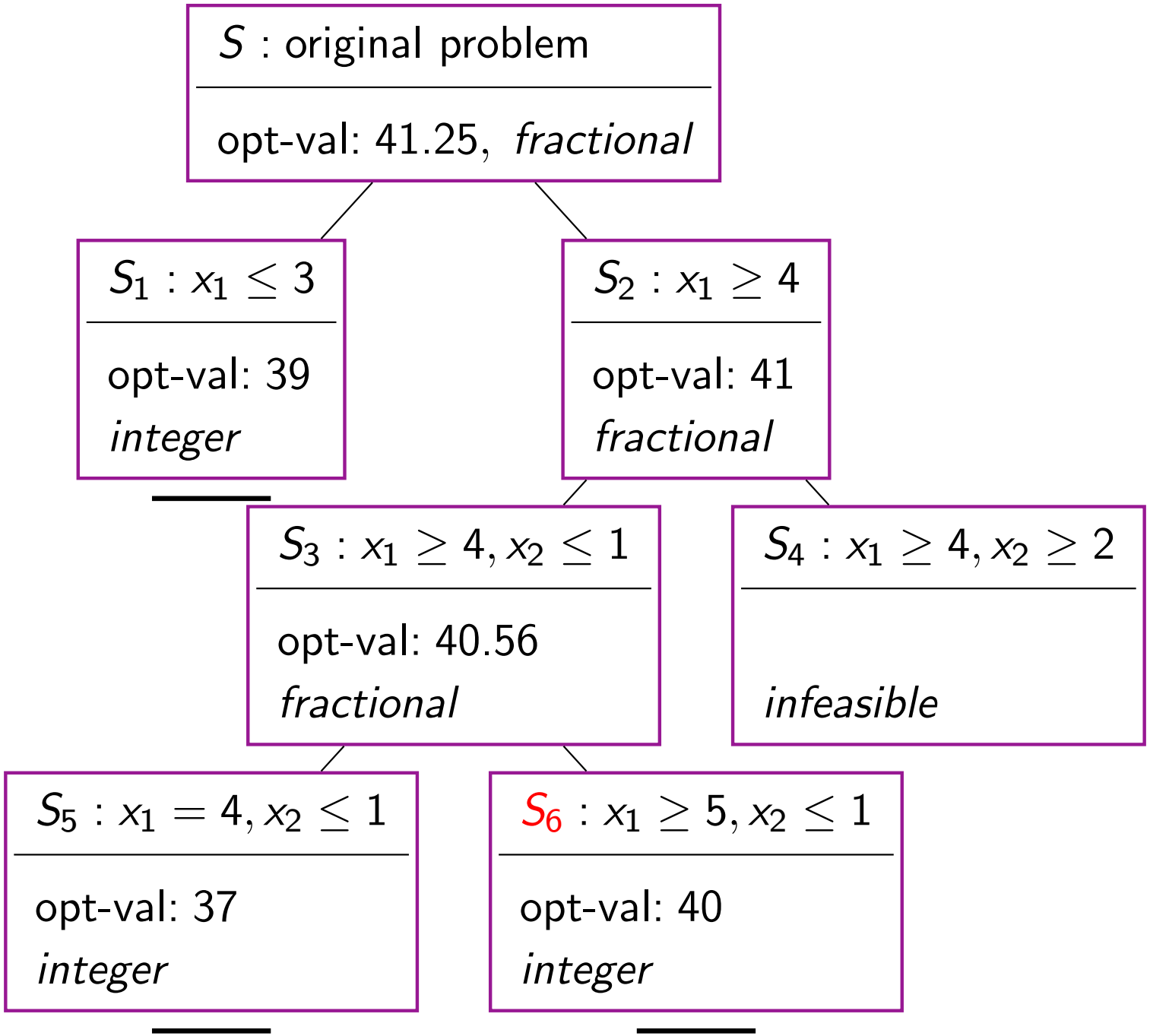
We solve (S3) and the optimal solution is $(40/9, 1)$, we have to do further branching:

- Add a constraint $x_1 \leq 4 \rightsquigarrow$ (S5).
- Add a constraint $x_1 \geq 5 \rightsquigarrow$ (S6).

we have constraint $x_1 \geq 4$

For (S5), x_1 has to be 4 and the optimal solution is $(4, 1)$ with objective value 37 (already integer, so do not need to do further branching).

For (S6), the optimal solution is $(5, 0)$, the objective value is 40.


S : original problem
opt-val: 41.25, *fractional*
$S_1 : x_1 \leq 3$
opt-val: 39
integer
$S_2 : x_1 \geq 4$
opt-val: 41
fractional
$S_3 : x_1 \geq 4, x_2 \leq 1$
opt-val: 40.56
fractional
$S_4 : x_1 \geq 4, x_2 \geq 2$
infeasible
$S_5 : x_1 = 4, x_2 \leq 1$
opt-val: 37
integer
$S_6 : x_1 \geq 5, x_2 \leq 1$
opt-val: 40
integer

Branch-and-Bound Method: Summary

High-level idea:

- **Branching:** Divide the feasible region into smaller ones, solve each of them and combine them to find the optimal solution.
- **Bounding:** Use bounds (LP optimal value and feasible points) to reduce the number of branches we need to consider.

Branch-and-Bound Method: Overview

Branching Procedures:

1. Solve the LP relaxation.
   - If the optimal solution is an integer, then it is optimal.
   - Otherwise go to step 2.

Branch-and-Bound Method: Overview

Branching Procedures:

1. Solve the LP relaxation.
   - If the optimal solution is an integer, then it is optimal.
   - Otherwise go to step 2.
2. If the optimal solution to the LP relaxation is x^* and x_i^* is fractional, then branch the problem into the following two:
   - One with an added constraint that $x_i \leq \lfloor x_i^* \rfloor$.
   - One with an added constraint that $x_i \geq \lceil x_i^* \rceil$.

Branching Procedures:

1. Solve the LP relaxation.
   - If the optimal solution is an integer, then it is optimal.
   - Otherwise go to step 2.
2. If the optimal solution to the LP relaxation is x^* and x_i^* is fractional, then branch the problem into the following two:
   - One with an added constraint that $x_i \leq \lfloor x_i^* \rfloor$.
   - One with an added constraint that $x_i \geq \lceil x_i^* \rceil$.
3. For each of the two problems, use the same method to solve them, and get optimal sol. y_1^* and y_2^* with optimal value v_1^* and v_2^*.
   - Compare to obtain the optimal solution.

Bounding procedures (for maximization):

- Any LP relaxation solution can provide an upper bound for each node in the branching process.
- Any feasible point to the IP can provide a lower bound for the entire problem.

Bounding procedures (for maximization):

- Any LP relaxation solution can provide an upper bound for each node in the branching process.
- Any feasible point to the IP can provide a lower bound for the entire problem.

When the optimal value of the LP relaxation of this branch is less than the current lower bound, then we can discard this branch.

- No better solution can be obtained from further exploring this branch.

Bounding procedures (for maximization):

- Any LP relaxation solution can provide an upper bound for each node in the branching process.
- Any feasible point to the IP can provide a lower bound for the entire problem.

When the optimal value of the LP relaxation of this branch is less than the current lower bound, then we can discard this branch.

- No better solution can be obtained from further exploring this branch.

Bounding is very important for branch-and-bound, it is the key to make it efficient (and practical).

Order of Computing the Branches

When we perform branch-and-bound, we may have two choices at each step:

- In the example, if we compute (S2) first, then we get a non-integer solution and thus two branches.
- Then we need to decide if we want to continue with one of the new branches or try (S1) next.

Order of Computing the Branches

When we perform branch-and-bound, we may have two choices at each step:

- In the example, if we compute (S2) first, then we get a non-integer solution and thus two branches.
- Then we need to decide if we want to continue with one of the new branches or try (S1) next.

Basically, we need to decide if we want to go deep into one branch first or go wide to solve all problems on a given level.

Deep or Wide?

In the branch-and-bound algorithm, the best approach is to go deep into the tree, not to go wide:

- Most integer solutions lie deep in the tree. It is good to have integer feasible solutions early, so we can use it in the bounding procedure.
- It is also memory-efficient, since each LP is obtained from its parent by merely adding one constraint.
- It is also easier to code (recursion).

Complexity of Branch-and-Bound

Branch-and-bound is essentially an enumeration method.

- In the worst case, branch-and-bound may need to go through each feasible integer point in the region, which is exponential in the problem size.
- Remember there is no polynomial-time algorithm for IP.

Complexity of Branch-and-Bound

Branch-and-bound is essentially an enumeration method.

- In the worst case, branch-and-bound may need to go through each feasible integer point in the region, which is exponential in the problem size.
- Remember there is no polynomial-time algorithm for IP.

However, branch-and-bound does enumeration in a smart way and typically it only needs to visit a tiny fraction of all solutions.

- Much more efficient than explicitly enumerating the solutions.
- It is one of the most useful practical methods.

Today's lecture

- Total Unimodularity
- Branch-and-Bound

Next lecture

- More about Branch-and-Bound

Questions?